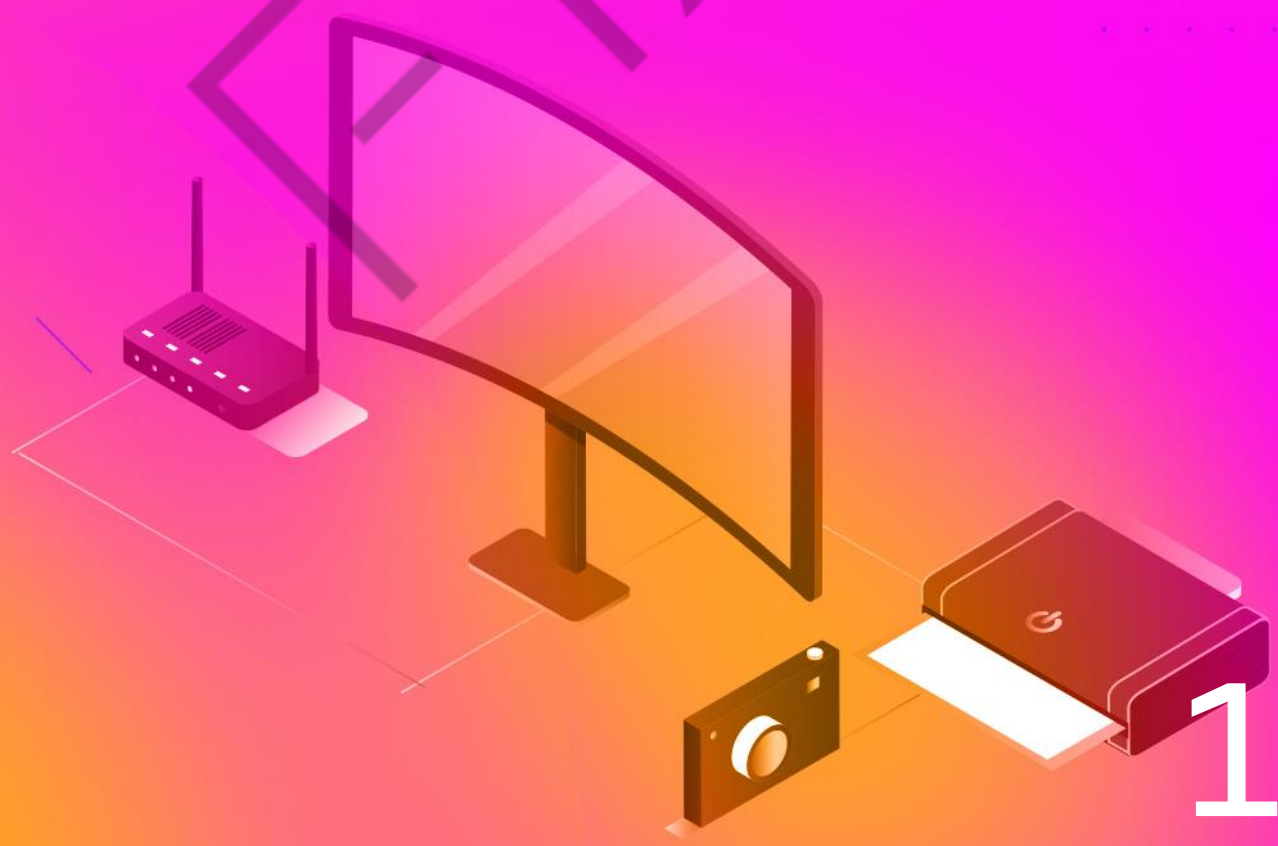
FIAP ON

SAFRA DO CONHECIMENTO
CIBERNÉTICO - PRAGAS VIRTUAIS

INTELIGÊNCIA NA REPRESENTAÇÃO DO CONHECIMENTO NO ESP32



12

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 1 - Ambiente básico do Wokwi.com com o Monitor Serial na parte cinza clara 7
Figura 2 - Monitor Serial do Arduino IDE mostrando o “Hello, ESP32!” 8
Figura 3 - Serial Plotter do Monitor Serial 8
Figura 4 - Impressão da temperatura e umidade no Serial Plotter do Wokwi.com....9
Figura 5 - Opções de conexão entre ESP32 e PC 12
Figura 6 - Gerenciador de Dispositivos do Windows, em destaque a porta COM que o ESP32 está conectado 13
Figura 7 - Arduino IDE com o ajuste da porta COM adequada 14
Figura 8 - Arduino IDE com o ajuste da porta COM adequada 15
Figura 9 - Tela do Wokwi ligando e desligando o LED via Monitor Serial 22
Figura 10 - Display real de 16x2.....38
Figura 11 - Tela do Wokwi solicitando a instalação da biblioteca I2C. Basta clicar no botão azul.....42
Figura 12 - Simulação do Wokwi usando LCD 16x2. Note que ele trabalha com 5V 42
Figura 13 - Contador de 0 a 1000 usando display para apresentar o conhecimento 43

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Exemplo de impressão de temperatura e umidade usando o millis() 10
Código-fonte 2 - Exemplo de como inicializa o Monitor Serial.....17
Código-fonte 3 - Exemplo de como entrar com apenas um caractere no Monitor Serial 18
Código-fonte 4 - Exemplo de como entrar com frases inteiras no Monitor Serial.....19
Código-fonte 5 - Exemplo de como entrar com números inteiros no Monitor Serial..20
Código-fonte 6 - Exemplo de como entrar com números decimais no Monitor Serial 20
Código-fonte 7 - Exemplo de como ligar e desligar um LED via Monitor Serial 21
Código-fonte 8 - Exemplo de como entrar com frases inteiras no Monitor Serial.....23
Código-fonte 9 - Exemplo de switch-case e entrada de dados via Monitor Serial.....26
Código-fonte 10 - Exemplo básico do uso de Serial.print() 30
Código-fonte 11 - Exemplo de como imprimir acentuação.....31
Código-fonte 12 - Exemplo de como imprimir em diferentes bases o mesmo número 32
Código-fonte 13 - Exemplo de impressão de casas decimais.....33
Código-fonte 14 - Exemplo de impressão de strings combinadas.....34
Código-fonte 15 - Exemplo de print() e println() 35
Código-fonte 16 - Exemplo de tabulação 36
Código-fonte 17 - Exemplo de tabulação 39
Código-fonte 18 - Exemplo de tabulação 44

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

1 INTELIGÊNCIA NA REPRESENTAÇÃO DO CONHECIMENTO NO ESP32 5
1.1 Introdução 5
1.2 Vantagens da representação do conhecimento 5
2 SERIAL PLOTTER DO MONITOR SERIAL 7
2.1 *Baud rate* do monitor ou Plotter Serial 11
2.2 *Baud rate* errado e seus efeitos 14
3 INPUT DE DADOS PELO MONITOR SERIAL.....16
3.1 Facilidade de interação e teste rápido.....16
3.2 Exemplos de entrada de dados no Monitor Serial.....17
3.3 Exemplos de entrada de dados no Monitor Serial com switch-case 23
4 COMO FORMATAR DADOS NO PRINT 28
4.1 Imprimindo acentuação no Monitor Serial 30
4.2 Formatando números inteiros.....31
4.3 Formatando números decimais (Float ou Double) 32
4.4 Formatando strings 33
4.5 Formatação de múltiplos valores com uma única chamada.....34
4.6 Alinhamento de colunas (Tabulação) 35
5 O USO DE DISPLAYS 37
5.1 Funcionamento de um LCD 16x2.....37
5.2 Pinagem e exemplo de código 38
5.3 Aplicações do LCD 16x2 em sistemas de IA.....44
CONCLUSÃO.....46
REFERÊNCIAS.....48

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1 INTELIGÊNCIA NA REPRESENTAÇÃO DO CONHECIMENTO NO ESP32

1.1 Introdução

Atualmente, o ESP32 é bastante utilizado em projetos de automação, sistemas embarcados e como coleta de dados para sistemas de Inteligência Artificial (IA). Entretanto, de nada adianta obter dados precisos de sensores ou realizar cálculos complexos sem uma forma eficaz de apresentar esses resultados ao usuário final.

A maneira como representamos, processamos e organizamos o conhecimento adquirido a partir do código-fonte no ESP32 é fundamental para a interpretação correta desses dados. Isso é importante para tomada de decisões rápidas no modelo de negócio, para dar prosseguimento no desenvolvimento do código-fonte, e para realizar monitoramento em tempo real em um sistema de IA.

Nesse capítulo, vamos aprender como podemos representar esse conhecimento em forma de display, como utilizar o gráfico nativo do Monitor Serial para plotar dados, como usar as opções do Serial.print(), e como entrar com dados no Monitor Serial para serem utilizados dentro do seu código-fonte.

1.2 Vantagens da representação do conhecimento

A representação de dados em sistemas com ESP32 envolve transformar informações brutas (como leituras de sensores, comandos de controle e resultados de processamento) em formas compreensíveis para o usuário final.

Isso pode ser feito através de gráficos, texto, interfaces mais interativas, como dashboards de front-end, e até mesmo codificar o ESP32 de forma organizada e estruturada para que outros desenvolvedores possam se inteirar do sistema ao observar as funções, classes e métodos do seu código.

Vejamos algumas das principais vantagens de representar o conhecimento de maneira visual e textual:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **Compreensão mais rápida:** a visualização de dados facilita a interpretação, permitindo ao usuário final perceber tendências, picos ou irregularidades rapidamente. Por exemplo, um gráfico de temperatura gerado pelo Monitor Serial ou exibido em um display pode indicar imediatamente quando uma máquina está superaquecendo. Com base nessas informações (segurança, negócio e manutenção), é possível tomar decisões importantes, como desligar o equipamento antes que ele trave ou pegue fogo, mesmo que você perca prazo, insumo, contrato etc.
- **Decisões informadas:** ao apresentar informações de forma visual, como gráficos ou números processados, o sistema permite que o usuário tome decisões com base em dados concretos. Um exemplo prático seria o uso de displays do tipo LCD (Liquid Crystal Display) para exibir o nível de um tanque de água, ajudando na decisão de encher ou esvaziar. Veremos mais detalhes sobre esse display ainda nesse capítulo.
- **Monitoramento em tempo real:** com o ESP32, o Monitor Serial ou displays conectados podem exibir dados em tempo real, ajudando o usuário a monitorar continuamente o comportamento do sistema. Isso é especialmente útil em ambientes críticos, como controle de temperatura em estufas agrícolas ou em processos industriais.
- **Customização e interatividade:** outra grande vantagem é a possibilidade de criar interfaces customizadas no front-end, conectando o ESP32 com páginas web ou apps mobile. Isso permite que o conhecimento seja representado em painéis interativos, ou seja, o usuário pode manipular e interagir com os dados recebidos, tornando o sistema mais flexível e dinâmico.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2 SERIAL PLOTTER DO MONITOR SERIAL

Considerando um exemplo de aplicação, pense no Monitor Serial do Wokwi ou do Arduino IDE (programa que grava ESP de verdade), que é aquela área onde você visualiza a mensagem “Hello, ESP32!” quando você roda um dos exemplos básicos de uso do ESP32, conforme mostra a figura “Ambiente básico do Wokwi.com com o Monitor Serial na parte cinza clara”.

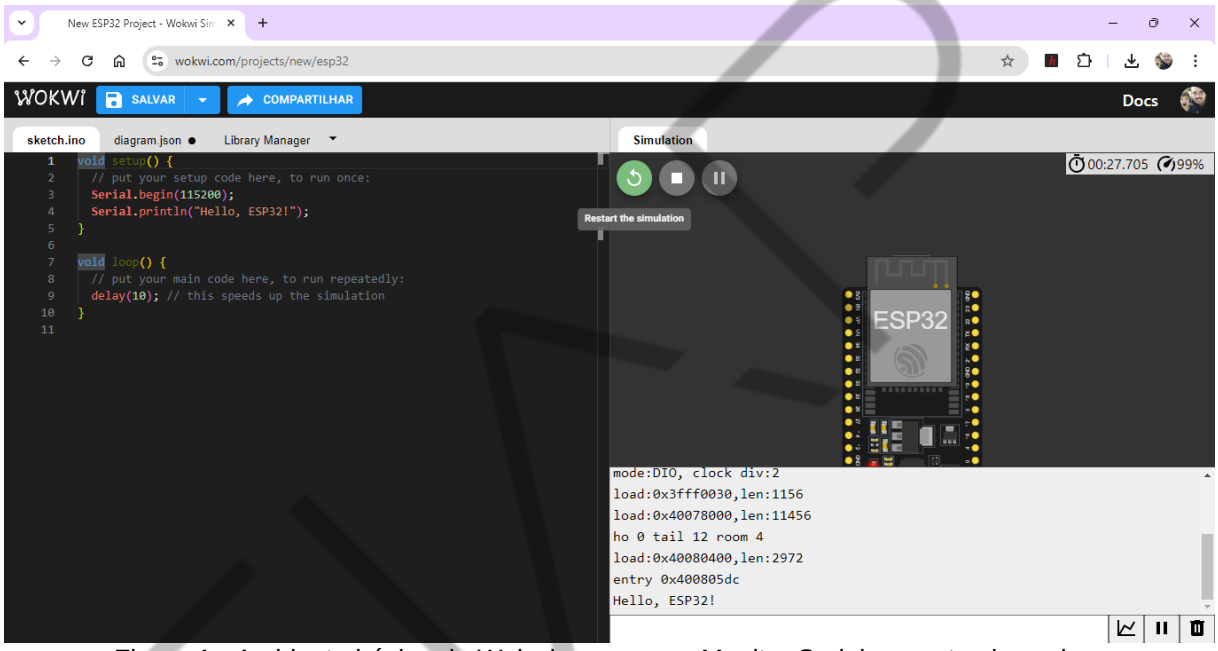


Figura 1 - Ambiente básico do Wokwi.com com o Monitor Serial na parte cinza clara
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

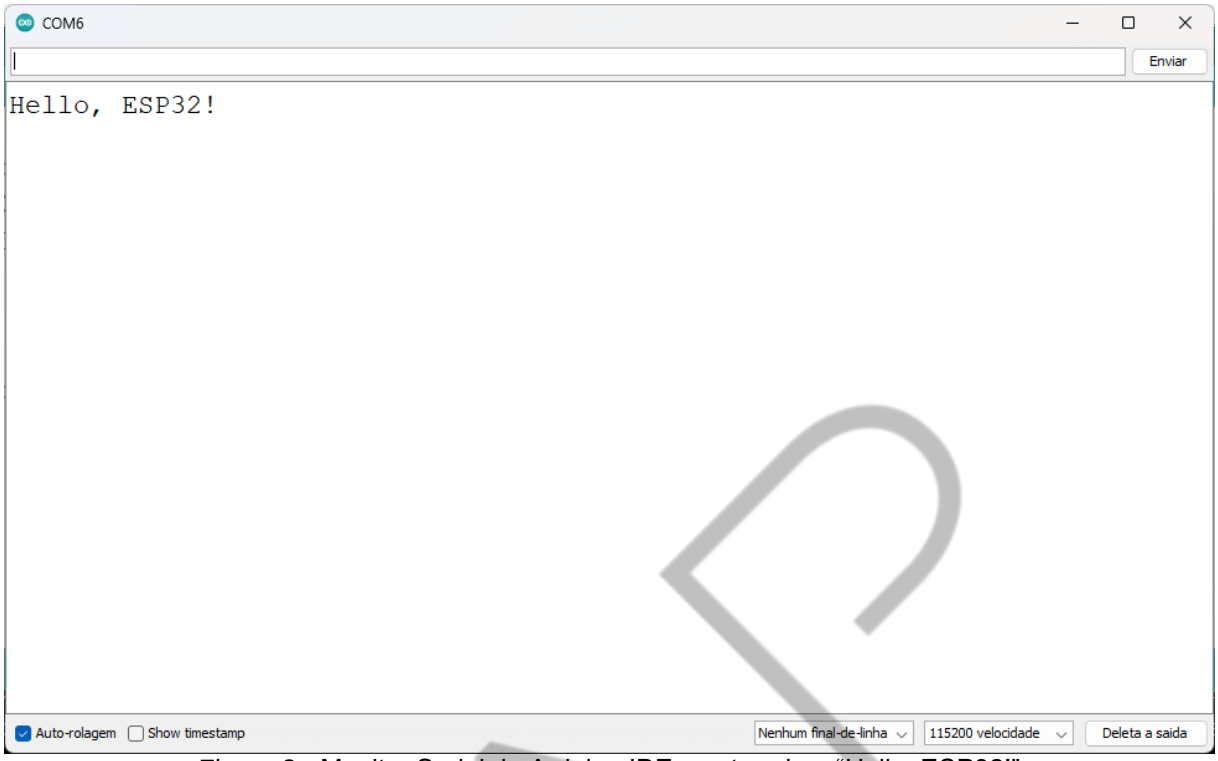


Figura 2 - Monitor Serial do Arduino IDE mostrando o “Hello, ESP32!”
Fonte: Elaborado pelo autor (2024)

Você sabia que é possível visualizar um gráfico a partir da impressão de valores no Monitor Serial? Para isso, basta você fazer um código-fonte que imprima os dados de temperatura e umidade no Monitor Serial, e clicar no botão que carrega o modo gráfico que está no canto inferior direito da tela do Wokwi.

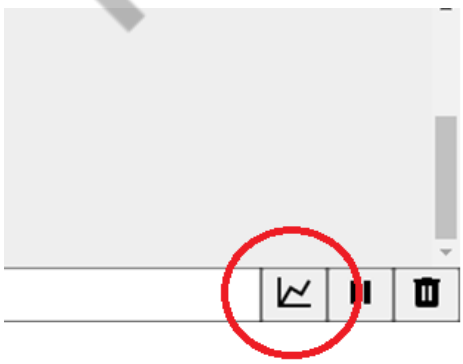


Figura 3 - Serial Plotter do Monitor Serial
Fonte: Elaborado pelo autor (2024)

Aproveitando a explicação, os demais ícones do canto direito inferior ao lado do Serial Plotter servem, respectivamente, para pausar ou continuar a simulação e a lixeira serve para apagar apenas os dados que já foram impressos no prompt do Monitor Serial. Se você guardou os dados dos sensores em uma variável em seu

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

código-fonte, estes não serão afetados. Observe como ficaria um sistema de monitoramento de temperatura usando Serial Plotter:

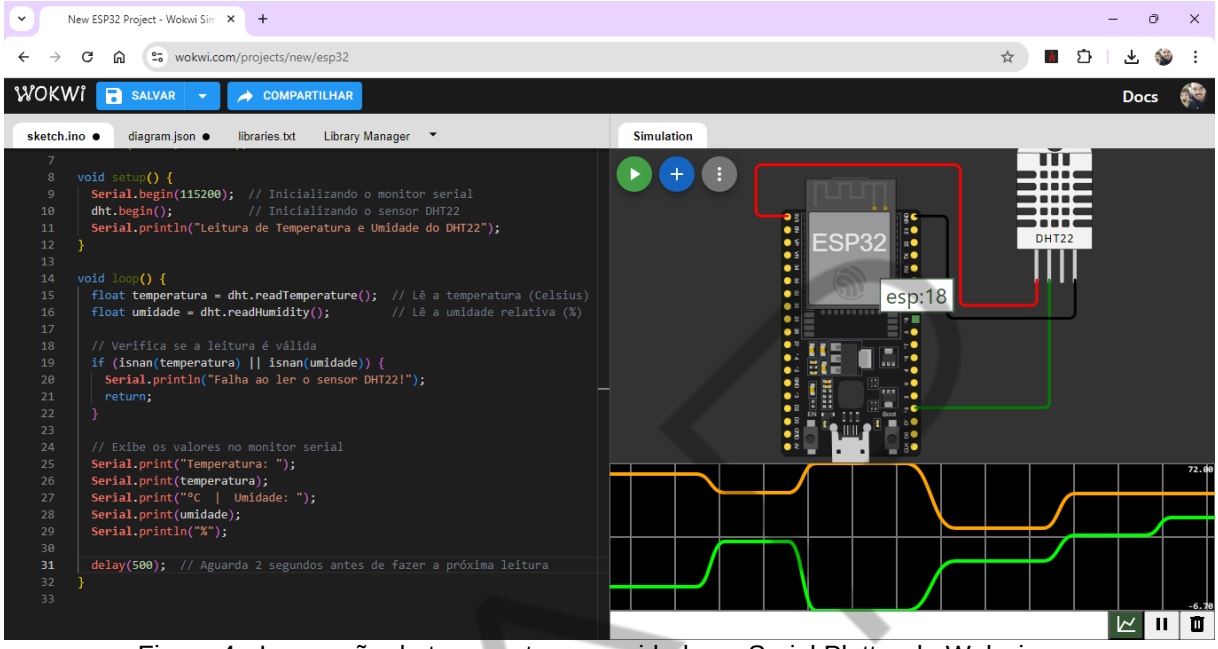


Figura 4 - Impressão da temperatura e umidade no Serial Plotter do Wokwi.com
Fonte: Elaborado pelo autor (2024)

O mesmo aconteceria se você estiver usando o Arduino IDE, isto é, você vai conseguir imprimir dados utilizando o modo gráfico do Monitor Serial, que é o Plotter Serial. Para abrir o Plotter Serial no Arduino IDE, vá no menu “Ferramentas”, e clique em “Plotter Serial”.

O código a seguir é responsável por ler os dados do DHT22 e imprimir os dados de temperatura e umidade.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
#include "DHT.h"
#define DHTPIN 15 // Defina o pino de dados do DHT22 (no Wokwi,
pode ser o pino 15)
#define DHTTYPE DHT22   // Definindo o tipo de sensor como DHT22

DHT dht(DHTPIN, DHTTYPE);

unsigned long ultimoTempo = 0;  // Variável para armazenar o
último tempo da leitura
const long intervalo = 500;    // Intervalo de 500 milissegundos
(0,5 segundos)

void setup() {
     Serial.begin(115200);  // Inicializando o monitor serial
     dht.begin();           // Inicializando o sensor DHT22
     Serial.println("Leitura  de  Temperatura  e  Umidade  do
DHT22");
}

void loop() {
// Verifica o tempo atual
     unsigned long tempoAtual = millis();

     // Se o tempo passado for maior que o intervalo, faz a
leitura
     if (tempoAtual - ultimoTempo >= intervalo) {
        ultimoTempo = tempoAtual;  // Atualiza o tempo da última
leitura

        // Lê a temperatura e a umidade
        float  temperatura  =  dht.readTemperature();  //  Lê  a
temperatura (Celsius)
        float  umidade  =  dht.readHumidity();  //  Lê  a  umidade
relativa (%)

        // Verifica se a leitura é válida
        if (isnan(temperatura) || isnan(umidade)) {
           Serial.println("Falha ao ler o sensor DHT22!");
           return;
        }

        // Exibe os valores no monitor serial
        Serial.print("Temperatura: ");
        Serial.print(temperatura);
        Serial.print("°C  |  Umidade: ");
        Serial.print(umidade);
        Serial.println("%");
     }
}
```

Código-fonte 1 - Exemplo de impressão de temperatura e umidade usando o millis()
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2.1 *Baud rate* do monitor ou Plotter Serial

A *baud rate*, ou taxa de baud, é uma medida da quantidade de mudanças em um sinal que se propagam por um meio de transmissão em um determinado período de tempo. É um termo utilizado na área de telecomunicações e informática para se referir à taxa de transmissão de dados em um canal de comunicação.

Em outras palavras, a *baud rate* é a medida em bauds por segundo (bps), e representa o número de vezes que um sinal elétrico é modulado ou alterado em um segundo.

A taxa de baud pode ser maior ou menor do que a taxa de bits, que é o número de bits por segundo que o usuário pode enviar pelo sistema de transmissão. Quando há exatamente dois símbolos no sistema (normalmente 0 e 1), então baud e bits *per second* são equivalentes.

E no que isso faz sentido? Tanto o Monitor Serial quanto o Plotter Serial possuem um *baud rate* que precisamos respeitar. Os valores possíveis são: 300, 1.200, 2.400, 4.800, 9.600, 19.200, 38.400, 57.600, 74.880, 115.200, 230.400, 250.000, 500.000, 1.000.000, 2.000.000 bps.

Quais são os valores mais empregados? 9600 e 115200 bps (**não use pontos decimais**) porque usamos hardware simples para nos comunicar com o ESP. Vale anotar que alguns dispositivos de Bluetooth (HC-05) usam 38400 bps de fábrica.

Portanto, ao iniciar o Serial.begin(9600) ou Serial.begin(115200) é exatamente esse *baud rate* que você está informando para o seu sistema do ESP32 adotar quando ele for se comunicar com o seu PC via porta USB.

A figura “Opções de conexão entre ESP32 e PC” mostra como são as duas opções de conexão entre o ESP32 e o seu computador (usando uma porta USB qualquer).

- **Opção 1:** Download / Log Ouput é a versão mais simples, e a mais usual, só precisa de um cabo de dados USB. Contudo, quando atua com velocidades superiores a 115200, pode acontecer o corrompimento ou perda de dados na transmissão devido ao afogamento do buffer.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **Opção 2:** AT command / response é a versão que permite *baud rates* rápidos, como os valores superiores a 500.000. Essa versão é pouco utilizada, mas é bom saber que ela existe quando projetos requerem *baud rates* superiores a 500000. Vale mencionar que projetos necessitam velocidades superiores quando enviam muitos dados para o PC, então, para não perder nenhum bit ou evento por afogamento do buffer, exige-se o controle de fluxo de bits por meio dos pinos CTS (Clear to Send) e RTS (Request to Send).

É necessário um bom cabo de dados, ou seja, deve-se atentar a qualidade dos cabos, pois vários cabos USB-C ou microUSB vendidos em lojas de artigos eletrônicos são apenas de alimentação, e por isso, eles não possuem os fios internos para a parte de dados, e consequentemente, seu ESP32 nunca funcionará ou terá sucesso no download de códigos-fontes a partir do Arduino IDE.

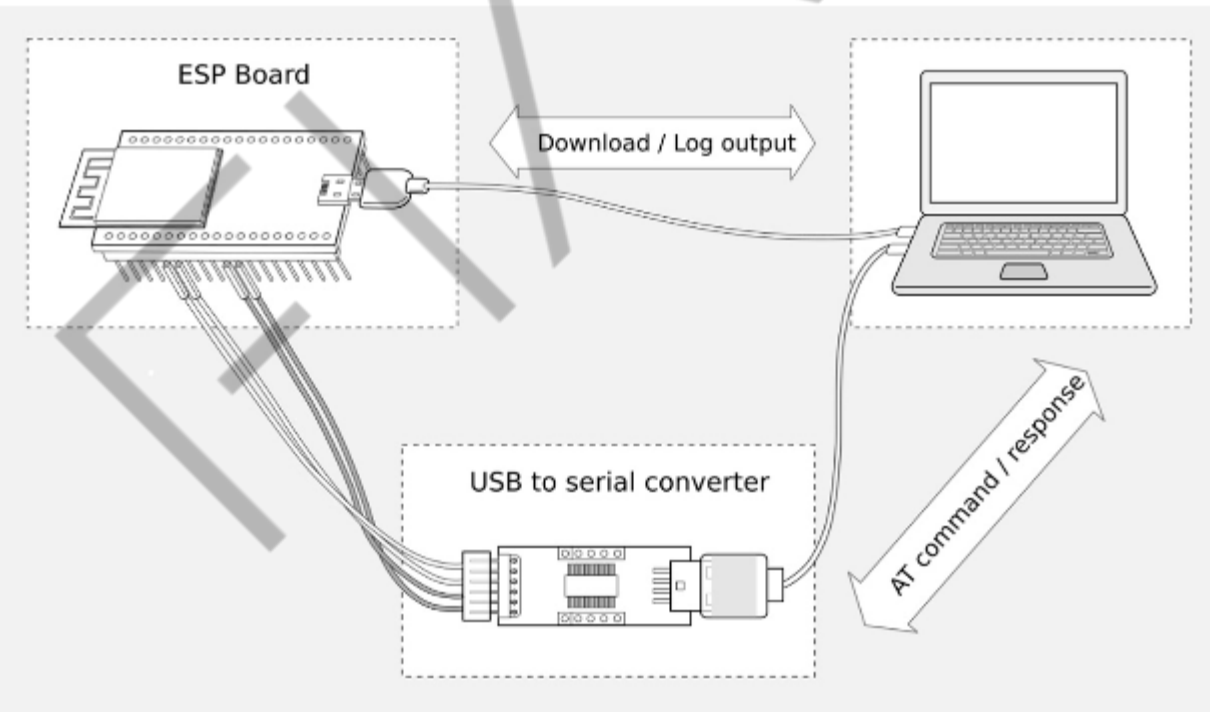


Figura 5 - Opções de conexão entre ESP32 e PC
Fonte: Espressif (2024)

Para se certificar que o cabo de dados que você pretende usar no seu ESP32 real seja de dados, faça o seguinte teste:

a) Conecte o ESP32 no seu computador;

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

b) Se for Windows, abra o Gerenciador de Dispositivos e observe a opção “portas”, conforme a figura a “Gerenciador de Dispositivos do Windows, em destaque a porta COM que o ESP32 está conectado”. Veja se aparece a descrição do tipo “Silicon Labs CP210... (COM6)” ou qualquer outro número no COM. Cada computador possui um número COM, geralmente menor que 12. Você deverá apontar o número COM no seu Arduino IDE quando for gravar um código-fonte no ESP32. Observe as figuras a seguir:

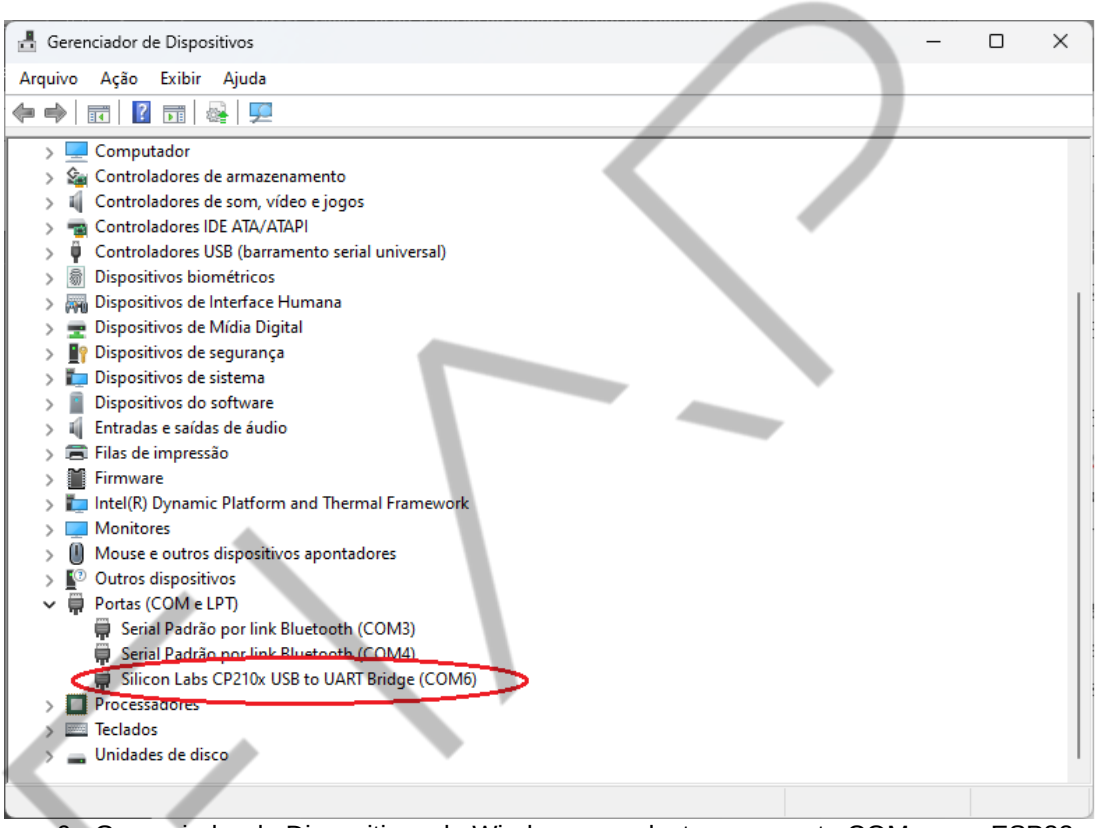


Figura 6 - Gerenciador de Dispositivos do Windows, em destaque a porta COM que o ESP32 está conectado
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

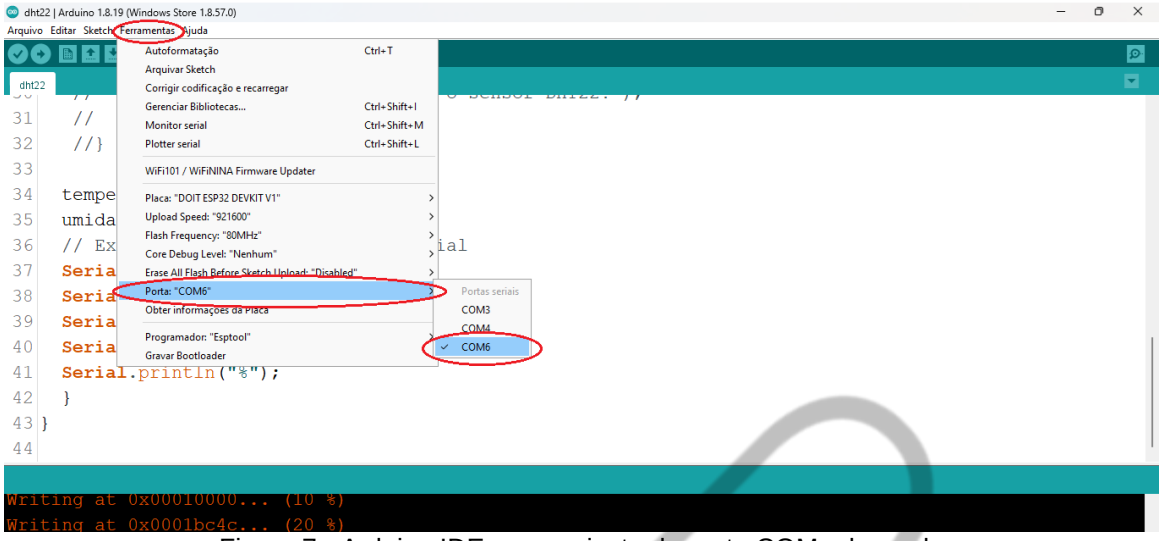


Figura 7 - Arduino IDE com o ajuste da porta COM adequada
Fonte: Elaborado pelo autor (2024)

c) Se o seu OS (Sistema Operacional) for o macOS, você pode descobrir a porta à qual o ESP32 está conectado usando o “Terminal”. Para isso, abra o “Terminal”, vá ao "Launchpad" ou use a pesquisa Spotlight (“Cmd + Espaço”) e procure por "Terminal". Nele, digite o seguinte comando para listar todos os dispositivos seriais conectados: ls /dev/cu.*

Esse comando mostrará uma lista de portas seriais disponíveis no sistema. O ESP32 geralmente aparece com um nome como “/dev/cu.SLAB_USBtoUART” ou algo semelhante a “cu.usbserial-*”. Conecte o ESP32 e rode o comando novamente. Se você ainda não conectou o ESP32 ao iMac, conecte agora e execute novamente o comando para identificar a nova porta que aparece na lista. Essa será a Porta Serial do ESP32. No Arduino IDE, vá em “Ferramentas > Porta” e selecione a porta correspondente que você identificou no Terminal.

2.2 *Baud rate* errado e seus efeitos

Quando o *baud rate* do Monitor Serial não está alinhado com o *baud rate* configurado no código no Serial.begin(), o efeito mais comum é que os dados exibidos no Monitor Serial aparecem como um monte de símbolos e caracteres sem sentido, como: ¬!@#%$^&*()+_={}[]:";'<>?,./~

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Isso acontece porque os bits estão sendo lidos em velocidades diferentes, causando erros de leitura. Fique tranquilo pois isso não estraga a sua placa ou o seu computador.

Outro efeito é a leitura de dados corrompida. Quando existe alguma diferença nos *baud rates*, os dados que o ESP32 tenta enviar ou receber podem ser interpretados incorretamente, resultando na exibição de valores errados ou incompletos, mesmo que esteja fazendo “sentido” para o usuário, estarão errados. E se você estiver tentando depurar o seu código ou monitorar leituras de sensores, será impossível entender as informações exibidas no Monitor Serial se os *baud rates* estiverem diferentes, pois os dados exibidos não farão sentido.

Para resolver o problema dos caracteres malucos, acerte o *baud rate* no final da tela do Monitor Serial (9600 ou 115200) na barra inferior ou ajuste o valor na função Serial.begin(9600) ou Serial.begin(115200) no seu código-fonte.

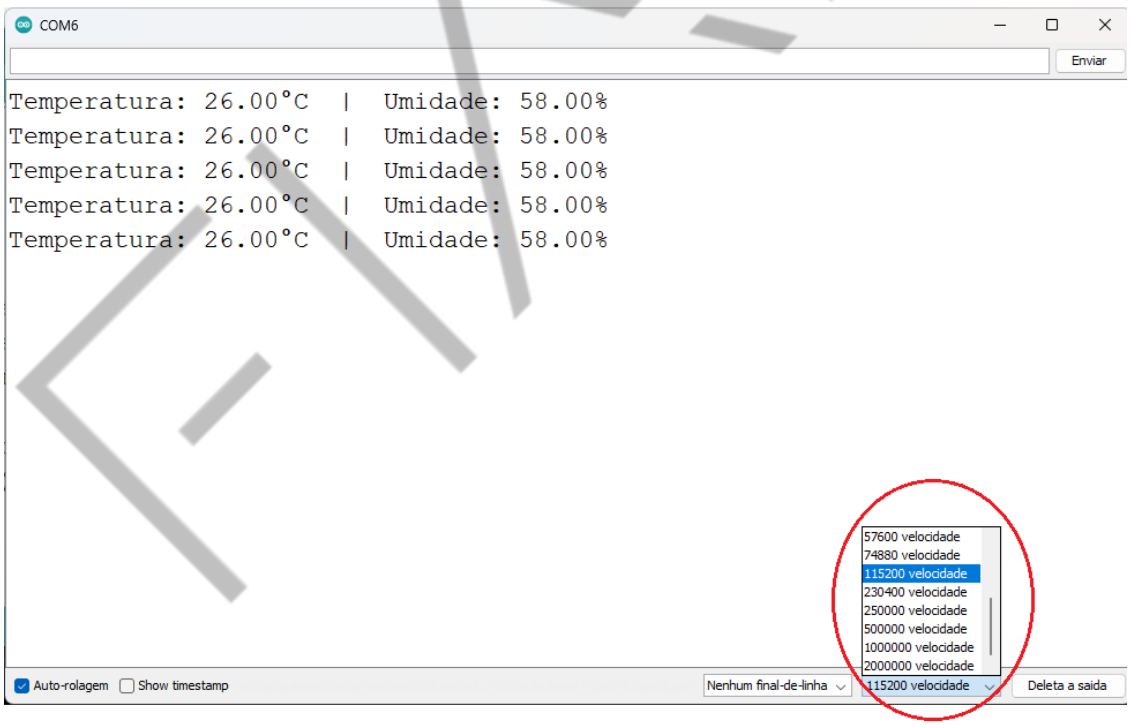


Figura 8 - Arduino IDE com o ajuste da porta COM adequada
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3 INPUT DE DADOS PELO MONITOR SERIAL

Em projetos de sistemas de IA utilizando o ESP32, o Monitor Serial desempenha um papel crucial na interação com o usuário e no ajuste dinâmico de parâmetros, especialmente durante o desenvolvimento e teste.

A entrada de dados via Monitor Serial permite que os desenvolvedores interajam com o sistema de maneira prática e direta, facilitando o ajuste de modelos, a coleta de dados e o controle de funcionalidades sem a necessidade de interfaces complexas ou reprogramações frequentes.

3.1 Facilidade de interação e teste rápido

Ao utilizar o Monitor Serial, você pode enviar comandos e ajustar parâmetros do sistema em tempo real. Isso é particularmente útil em sistemas onde muitas vezes é necessário experimentar diferentes valores para variáveis, como velocidade de leitura de sensores, alteração do valor do millis — usado no lugar do delay() —, limites de decisão ou até parâmetros de sensores.

Como sabemos, o ESP32 está crescendo na área dos modelos de IA e, por isso, sua principal função hoje é coletar dados por meio de sensores e alimentar os modelos, não rodar o modelo propriamente dito.

Com o Monitor Serial, o(a) desenvolvedor(a) pode fazer esses ajustes rapidamente, sem a necessidade de reescrever e carregar o código constantemente.

Por exemplo, em um sistema de IA que utiliza um modelo de Machine Learning em um computador comum, com o ESP32 como parceiro para trazer dados e classificar dados de sensores, é possível ajustar as condições de ativação ou limites de confiança diretamente pelo Monitor Serial, tornando o processo de ajuste mais eficiente. Essas informações de ativação ou limite podem ser transmitidas via Wi-Fi para o computador em que está rodando o modelo.

O Monitor Serial é uma interface padrão e simples de ser utilizada, pois já está pronta. Não é necessário configurar interfaces gráficas complexas ou investir em hardware adicional para interagir com o sistema.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Isso permite que desenvolvedores e usuários finais manipulem o comportamento da IA utilizando comandos simples via texto, como se estivessem interagindo com uma linha de comando.

Além disso, para relembrar, qualquer plataforma que possa se comunicar com o ESP32 (como o Arduino IDE ou ferramentas como o Wokwi) já possui um Monitor Serial embutido, simplificando ainda mais a implementação.

3.2 Exemplos de entrada de dados no Monitor Serial

O Monitor Serial é uma ferramenta valiosa para entrar dados em projetos com ESP32, permitindo que o desenvolvedor ou o usuário interaja com o sistema sem a necessidade de hardware extra ou ficar gravando várias mudanças de valores de variáveis.

Através do Monitor Serial, podemos enviar comandos, números e até strings completas, que o ESP32 pode processar para ajustar comportamentos, realizar cálculos ou executar funções específicas.

Antes de podermos entrar dados no Monitor Serial, é preciso inicializar a comunicação serial no ESP32 utilizando a função “Serial.begin()”. Como visto, o “Serial.begin()” define a taxa de transferência de dados (*baud rate*), que precisa ser configurada corretamente no ESP32 e no Monitor Serial para garantir que os dados sejam lidos e interpretados corretamente.

```
void setup() {
  Serial.begin(115200);  // Inicializa a comunicação serial com
baud rate de 115200
  Serial.println("ESP32  pronto  para  receber  dados!");    //
Mensagem inicial
}
```

Código-fonte 2 - Exemplo de como inicializa o Monitor Serial
Fonte: Elaborado pelo autor (2024)

Aqui, configuramos o *baud rate* para 115200 bps, que é uma taxa comum e rápida de transferência. Para reforçar, não se esqueça de configurar o mesmo *baud rate* no Monitor Serial para evitar problemas de comunicação.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

A maneira mais básica de capturar dados do Monitor Serial é usando a função “Serial.read()”, que lê um caractere (byte) por vez.

```
void setup() {
  Serial.begin(115200);
  Serial.println("Digite um caractere:");
}

void loop() {
  if (Serial.available() > 0) {    //Verifica se há dados
disponíveis no monitor serial
    char entrada = Serial.read(); // Lê o caractere
    Serial.print("Você digitou: ");
    Serial.println(entrada);      // Exibe o caractere digitado
  }
}
```

Código-fonte 3 - Exemplo de como entrar com apenas um caractere no Monitor Serial
Fonte: Elaborado pelo autor (2024)

Explicação do código:

- Em Serial.available(), o código verifica se há bytes disponíveis para leitura.
- Em Serial.read(), lê-se um byte de cada vez (um caractere).

Este método é útil quando se deseja processar entrada simples baseada em comandos de um caractere, como “y” para "sim" e “n” para "não".

Agora, vamos avançar um pouco mais. Se o seu projeto precisa capturar palavras ou frases completas (strings), você pode utilizar o “Serial.readString()”, que lê até encontrar um caractere de nova linha (“\n” ou “\r\n”), indicando que o usuário terminou de digitar.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
void setup() {
  Serial.begin(115200);
  Serial.println("Digite uma frase:");
}

void loop() {
  if (Serial.available() > 0) {
    String entrada = Serial.readString(); // Lê string completa
    Serial.print("Você digitou: ");
    Serial.println(entrada); // Exibe a string digitada
  }
}
```

Código-fonte 4 - Exemplo de como entrar com frases inteiras no Monitor Serial
Fonte: Elaborado pelo autor (2024)

Explicação:

- O ESP32 espera até que o usuário termine de digitar e pressione "Enter".
- A função “Serial.readString()” captura a linha completa de texto digitada no Monitor Serial.
- Se for usar o Wokwi para testar, pode existir um pequeno tempo de processamento após digitar o “Enter”.
- Quando usar o ESP32 real, não existe atraso de processamento após digitar o “Enter”.

Contudo, isso é útil para entradas maiores, como quando o usuário precisa digitar uma sequência de comandos ou enviar parâmetros mais complexos.

Agora, imagine que você precisa tratar o dado que o usuário que vai entrar. Por exemplo, seu projeto precisa capturar números inteiros e número quebrados (os famosos números com casas decimais), e para isso, o ESP32 oferece funções dedicadas, como “Serial.parseInt()” e “Serial.parseFloat()”.

Exemplo de como ler um número inteiro com “Serial.parseInt()”:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
void setup() {
  Serial.begin(115200);
  Serial.println("Digite um número inteiro:");
}

void loop() {
  if (Serial.available() > 0) {
    int numero = Serial.parseInt();// Lê o número inteiro
    Serial.print("Você digitou o número: ");
    Serial.println(numero);// Exibe o número digitado
  }
}
```

Código-fonte 5 - Exemplo de como entrar com números inteiros no Monitor Serial
Fonte: Elaborado pelo autor (2024)

Exemplo de como ler um número decimal com “Serial.parseFloat()”:

```
void setup() {
  Serial.begin(115200);
  Serial.println("Digite um número decimal:");
}

void loop() {
  if (Serial.available() > 0) {
    float numero = Serial.parseFloat();  // Lê o número decimal
    Serial.print("Você digitou o número: ");
    Serial.println(numero);// Exibe o número decimal digitado
  }
}
```

Código-fonte 6 - Exemplo de como entrar com números decimais no Monitor Serial
Fonte: Elaborado pelo autor (2024)

Essas funções são úteis em projetos que exigem entrada numérica para cálculos, como definir parâmetros de controle ou valores de sensores.

Ainda nesse capítulo, vamos discutir como controlar o número de casas decimais que desejamos na impressão. Por enquanto, esse exemplo está com duas casas decimais como padrão.

Agora vamos dar um passo adiante e criar um sistema onde o usuário pode enviar comandos específicos pelo Monitor Serial, e o ESP32 executa ações diferentes com base nesses comandos.

Considere que você tenha um LED conectado ao pino GPIO 2 do ESP32. Você pode usar o Monitor Serial para enviar comandos para ligar ou desligar o LED.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
#define LED_PIN 23  // Definindo o pino do LED

void setup() {
  pinMode(LED_PIN, OUTPUT);  // Define o pino do LED como saída
  Serial.begin(115200);
  Serial.println("Digite 'on' para ligar o LED e 'off' para
desligar.");
}

void loop() {
  if (Serial.available() > 0) {
    String comando = Serial.readString();  // Lê o comando
digitado
    comando.trim();  // Remove espaços em branco e quebras de
linha

    if (comando == "on") {
      digitalWrite(LED_PIN, HIGH);  // Liga o LED
      Serial.println("LED ligado!");
    } else if (comando == "off") {
      digitalWrite(LED_PIN, LOW);  // Desliga o LED
      Serial.println("LED desligado!");
    } else {
      Serial.println("Comando inválido. Tente 'on' ou 'off'.");
    }
  }
}
```

Código-fonte 7 - Exemplo de como ligar e desligar um LED via Monitor Serial
Fonte: Elaborado pelo autor (2024)

Explicação:

- O usuário pode digitar “on” para ligar o LED e “off” para desligá-lo.
- O comando.trim() remove qualquer espaço em branco adicional ou quebras de linha ao final da string, garantindo que apenas o comando em si seja processado.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

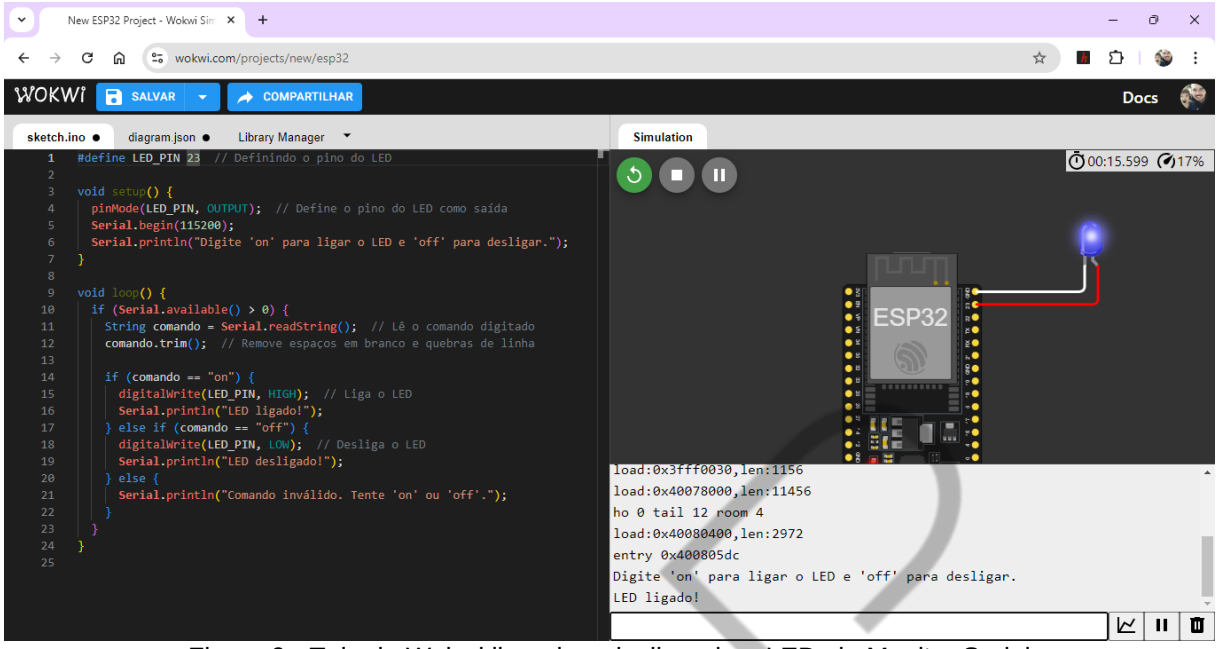


Figura 9 - Tela do Wokwi ligando e desligando o LED via Monitor Serial
Fonte: Elaborado pelo autor (2024)

Observação: note que o LED azul do circuito não precisa de resistor em série, pois o LED azul trabalha com tensão de 3,3V com folga.

Esse padrão de uso do Monitor Serial para capturar comandos e processá-los é bastante comum na fase de testes do seu projeto, onde é necessário validar o controle de dispositivos antes de colocá-lo em produção.

Aqui está um exemplo completo com base no if-else de um sistema onde o usuário pode digitar números inteiros e o ESP32 executa uma ação específica com base no valor.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
void setup() {
  Serial.begin(115200);
  Serial.println("Digite um número inteiro:");
}

void loop() {
  if (Serial.available() > 0) {
    int numero = Serial.parseInt();  // Lê um número inteiro

    if (numero == 1) {
      Serial.println("Você digitou 1! Acionando função 1...");
      // Execute a função 1
    }
    else if (numero == 2) {
      Serial.println("Você digitou 2! Acionando função 2...");
      // Execute a função 2
    }
    else {
      Serial.println("Número desconhecido! Tente novamente.");
    }
  }
}
```

Código-fonte 8 - Exemplo de como entrar com frases inteiras no Monitor Serial
Fonte: Elaborado pelo autor (2024)

Para finalizar, vamos a um exemplo prático utilizando “switch-case” para capturar números digitados pelo usuário no Monitor Serial, e o ESP32 responde com ações específicas. Esse “switch-case” é uma excelente alternativa para os vários if-else.

3.3 Exemplos de entrada de dados no Monitor Serial com switch-case

O switch-case é uma estrutura de controle utilizada para decidir entre várias opções com base no valor de uma variável. No ESP32, essa estrutura é particularmente útil em casos em que há múltiplas opções e o comportamento do programa precisa mudar de acordo com o valor de entrada do usuário ou o estado de uma variável. Algumas vantagens de seu emprego:

a) A estrutura “switch-case” facilita a organização e a leitura do código, especialmente quando você precisa tratar múltiplos cenários com base em uma única variável. Isso é mais claro do que o uso de várias instruções “if-else” consecutivas, tornando o código mais fácil de entender e manter. Por exemplo,

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

imagine um sistema de controle para um projeto de IoT (Internet of Things) onde você precisa escolher entre diferentes ações (ligar luzes, sensores, motores, entre outros) com base no comando de entrada. Usar o “switch-case” permite que você defina esses diferentes comportamentos de maneira organizada.

b) O “switch-case” pode ser mais eficiente em termos de desempenho do que uma sequência de “if-else” em situações em que há muitas comparações. Isso ocorre porque o “switch-case” compila de forma mais otimizada, enquanto uma série de “if-else” pode exigir múltiplas verificações lineares. No caso do ESP32, no qual a capacidade de processamento e memória é valiosa, essa eficiência pode ser importante em programas complexos.

c) É facilmente escalável, permitindo adicionar novos casos sem complicar o código. Se você estiver desenvolvendo um projeto que pode crescer e incluir mais funcionalidades, como diferentes modos de operação em um sistema de controle de automação, o “switch-case” permite incluir novos comandos sem alterar a lógica já existente.

d) Oferece um controle de fluxo claro e direto. Cada caso é claramente definido, e o uso de “break” evita que os outros casos sejam executados. Isso garante que o código execute exatamente o comportamento correspondente ao valor da variável.

Porém, como nem tudo são flores, existem as desvantagens:

a) Um dos maiores limites do “switch-case” é que ele só funciona com tipos de dados inteiros ou caracteres (no caso do ESP32, tipos como “int” e suas derivações, int8_t, int16_t, entre outros, e o “char”). Isso significa que você não pode usá-lo diretamente para comparar strings ou valores booleanos sem convertê-los. Se precisar comparar strings (como comandos digitados no Monitor Serial), você terá que usar “if-else”, o que pode complicar a lógica. Por exemplo, se você precisa processar strings como "ligar" e "desligar" (comandos enviados via serial), não pode usar diretamente o “switch-case”. Nesse caso, o “if-else” seria mais adequado.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

b) Se você esquecer de colocar a instrução “break” após cada “*case*”, o programa continuará executando os próximos casos (fenômeno conhecido como “*fall-through*”), o que pode gerar comportamentos inesperados. Isso pode ser um problema, especialmente em sistemas críticos, onde ações erradas podem levar a falhas. Por exemplo, se você esquecer de adicionar “break” após um caso que liga um dispositivo, o próximo caso pode ser executado indevidamente, desativando o outro dispositivo sem intenção.

c) O “switch-case” não é ideal para comparações que exigem condições complexas, como verificações lógicas ou múltiplas variáveis ao mesmo tempo. Para isso, o uso de “if-else” ou operadores lógicos seria mais adequado.

d) Se você precisar lidar com intervalos de valores (por exemplo, verificar se um número está entre 1 e 10), o “switch-case” não é muito prático. O “if-else” permite criar condições para esses intervalos de forma mais direta.

Neste exemplo com switch-case, vamos imaginar que estamos desenvolvendo um sistema de IoT para controle de dispositivos inteligentes em uma casa ou indústria ou fazenda, como lâmpadas, ar-condicionado e sensores. O ESP32, que é o “cérebro” do sistema, recebe comandos do usuário via Monitor Serial para controlar esses dispositivos. Em um sistema real, isso poderia ser conectado a sensores, motores, ou até mesmo sistemas baseados em IA para ajustar automaticamente as configurações com base nas entradas do usuário.

```
float temperatura = 24.5;  // Exemplo fictício de temperatura
void setup() {
  Serial.begin(115200);  // Inicializa a comunicação serial
  Serial.println("Sistema de Controle Inteligente Iniciado.");
  Serial.println("Digite   um   número   para   controlar   os
dispositivos:");
  Serial.println("1: Ligar lâmpada");
  Serial.println("2: Ligar ar-condicionado");
  Serial.println("3:   Verificar   status   do   sensor   de
temperatura");
  Serial.println("4: Acionar modo automático (IA)");
}

void loop() {
  if (Serial.available() > 0) {
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    String entrada = Serial.readString();  // Lê a entrada como
string
    entrada.trim();  // Remove espaços em branco e novas linhas

    int comando = entrada.toInt();  // Converte a string para
número inteiro

    switch (comando) {
      case 1:
        Serial.println("Lâmpada ligada!");
        // Aqui, no mundo real, você poderia controlar um relé
ou LED que liga a lâmpada
        break;
      case 2:
        Serial.println("Ar-condicionado ligado!");
        // No mundo real, você enviaria um comando para o
controle do ar-condicionado
        break;
      case 3:
        Serial.println("Lendo temperatura atual...");
        Serial.print("Temperatura: ");
        Serial.print(temperatura);
        Serial.println(" °C");
        break;
      case 4:
        Serial.println("Modo automático ativado! Usando IA para
otimizar o ambiente.");
        // Ativar a lógica de IA para ajustar as configurações
do ambiente
        break;
      default:
        Serial.println("Comando inválido! Tente novamente.");
        break;
    }
  }
}
```

Código-fonte 9 - Exemplo de switch-case e entrada de dados via Monitor Serial
Fonte: Elaborado pelo autor (2024)

Explicação:

- No loop(), usamos “Serial.available()” para verificar se o usuário digitou algo.
- Serial.readString(): lê a entrada completa, incluindo a parte invisível que é enviada quando o usuário pressiona "Enter".
- A função Serial.parseInt() não era capaz de lidar corretamente com o fim de linha, então substituímos por readString().

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- entrada.trim(): remove qualquer espaço em branco ou caractere extra de nova linha (\n ou \r\n) para garantir que estamos lidando apenas com o número digitado.
- entrada.toInt(): converte a string lida para um número inteiro. Assim, você pode usar esse número diretamente no switch-case. O “switch-case” permite escolher entre diferentes ações baseadas no valor do número digitado.
- Cada “*case*” corresponde a um comando específico:
  - 1 para ligar a lâmpada.
  - 2 para ligar o ar-condicionado.
  - 3 para verificar o sensor de temperatura.
  - 4 para ativar o modo automático com IA.
  - O “default” trata qualquer comando que não corresponda aos valores esperados, exibindo uma mensagem de erro.
- Para cada comando, uma mensagem é impressa no Monitor Serial informando ao usuário o que está acontecendo. No mundo real, os comandos poderiam acionar componentes físicos, como relés, LEDs, ou até mesmo sistemas baseados em IA.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4 COMO FORMATAR DADOS NO PRINT

O uso de Serial.print() no ESP32 é essencial para a comunicação com o usuário, depuração de código, e exibição de dados em tempo real. As opções de formatação dessa função permitem controlar como os dados (números, strings, e outros tipos de dados) são apresentados no Monitor Serial e em displays, facilitando o desenvolvimento e a análise do comportamento do sistema. No entanto, assim como qualquer ferramenta, essas opções apresentam vantagens e desvantagens que devem ser consideradas ao trabalhar com o ESP32.

Vejamos algumas vantagens das opções de formatação do Serial.print():

a) O “Serial.print()” permite exibir números inteiros, números decimais e strings, dando ao desenvolvedor liberdade para escolher a melhor forma de exibir informações. Por exemplo, é possível exibir números em bases numéricas diferentes (decimal, binário, hexadecimal) ou ajustar a precisão de números de ponto flutuante.

b) Ao exibir números decimais (float ou double), é possível especificar o número de casas decimais que serão exibidas, o que é útil em medições de alta precisão, como em sensores de temperatura ou umidade. Por exemplo, um sensor de temperatura pode precisar exibir apenas duas casas decimais (23.45 °C), sem sobrecarregar a tela com valores imprecisos ou irrelevantes.

c) O uso de múltiplos Serial.print() permite concatenar dados e strings em uma única linha de forma organizada. Além disso, o uso de caracteres de controle como tabulação (\t) e espaçamento ajuda a alinhar as saídas, tornando-as mais legíveis e claras. Por exemplo, exibir valores de sensores em colunas alinhadas facilita o monitoramento simultâneo de múltiplos dispositivos.

d) A formatação de saídas com Serial.print() facilita a depuração de código. O desenvolvedor pode exibir variáveis em diferentes etapas do programa para verificar se os valores estão corretos, ajudando a identificar erros rapidamente. Por exemplo, durante o desenvolvimento você pode exibir

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

variáveis de controle, tempo de execução ou estados de sensores para garantir que o código esteja funcionando conforme o esperado.

E as desvantagens da formatação do Serial.print()? Vejamos:

a) Embora o Serial.print() seja flexível com strings simples, concatenar strings com variáveis pode tornar o código mais complexo e difícil de gerenciar, especialmente quando se trabalha com grandes quantidades de dados ou formatação avançada. Por exemplo, concatenar strings e variáveis manualmente pode resultar em um código verboso e difícil de manter. Para saídas mais complexas, como mensagens formatadas ou tabelas, outras abordagens (como sprintf()) podem ser mais adequadas.

b) Comparado a funções como printf() em outras linguagens, o Serial.print() oferece um controle limitado sobre a formatação de dados. Ele não permite formatação avançada, como alinhamento de texto ou preenchimento com zeros à esquerda, o que pode ser útil em certas aplicações. Por exemplo, se você quiser exibir um número com zeros à esquerda (como 00123), precisará recorrer a métodos manuais, pois o Serial.print() não oferece suporte direto para isso.

c) Embora o Serial.print() seja eficiente para exibição de dados, seu uso excessivo dentro de loops críticos pode causar lentidão no processamento do ESP32. Enviar muitos dados para o Monitor Serial pode aumentar a latência do sistema, especialmente se a taxa de *baud rate* não for alta o suficiente para acompanhar o volume de dados. Outro momento que se deve evitar o uso do Serial.print() é dentro das interrupções do ESP32. Por exemplo, em sistemas de tempo real ou com muitos sensores, a repetição contínua de saídas pode comprometer o desempenho geral do sistema.

d) O Serial.print() não tem suporte para imprimir diretamente estruturas de dados complexas, como arrays ou objetos, sem a necessidade de transformar manualmente os dados em strings ou números individuais. Isso pode ser uma desvantagem quando o sistema exige o monitoramento de várias variáveis de uma só vez. Por exemplo, para exibir todos os valores de um array de sensores, você terá que iterar manualmente sobre o array e usar múltiplos Serial.print().

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4.1 Imprimindo acentuação no Monitor Serial

O uso da função “Serial.print()” (ou sua variante “Serial.println()”) é uma das maneiras mais comuns de exibir e depurar informações em projetos com ESP32. Mas qual é a diferença entre eles?

- Serial.print() exibe os dados no Monitor Serial sem adicionar uma nova linha no final. Assim, a próxima chamada de “Serial.print()” continuará na mesma linha.
- Serial.println() exibe os dados e adiciona automaticamente uma nova linha ao final, movendo o cursor do Monitor Serial para a linha seguinte. Isso é útil quando queremos exibir várias saídas de forma organizada, cada uma em sua própria linha.

Vejamos alguns exemplos básicos.

```
void setup() {
  Serial.begin(115200);
  Serial.print("Ola, ESP32! ");   // O cursor continua na mesma
linha
  Serial.println("Tudo bem?");    // O cursor vai para a próxima
linha
}

void loop() {
  // Código adicional
}
```

Código-fonte 10 - Exemplo básico do uso de Serial.print()
Fonte: Elaborado pelo autor (2024)

Saída esperada: Ola, ESP32! Tudo bem?

Infelizmente, o Serial.print() e as plataformas Arduino não suportam acentuação do português brasileiro de forma direta. Contudo, uma alternativa é usar o código ASCII estendido.

Por exemplo:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
void setup() {
  Serial.begin(115200);

  // Enviando diretamente os valores ASCII estendidos
  Serial.write(135); // ASCII estendido para 'ç'
  Serial.write(" ");
  Serial.write(130); // ASCII estendido para 'é'
}

void loop() {
  // Código adicional
}
```

Código-fonte 11 - Exemplo de como imprimir acentuação
Fonte: Elaborado pelo autor (2024)

4.2 Formatando números inteiros

Por padrão, o “Serial.print()” pode exibir números inteiros, mas também permite escolher em que “base numérica” você quer exibir esses números, sendo útil em projetos que envolvem manipulação de dados binários, hexadecimais ou outras representações numéricas. Exemplo de Bases Numéricas:

- Decimal (Base 10): a base padrão para exibir números inteiros.
- Binário (Base 2): exibe o número em formato binário.
- Hexadecimal (Base 16): exibe o número no formato hexadecimal.
- Octal (Base 8): exibe o número no formato octal.

Sintaxe: Serial.print(valor, base);

Exemplo de código:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
void setup() {
  Serial.begin(115200);
  int numero = 255;

  Serial.println("Formato Decimal: ");
  Serial.println(numero);  // Exibe em decimal (padrão)

  Serial.println("Formato Binário: ");
  Serial.println(numero, BIN);  // Exibe em binário

  Serial.println("Formato Hexadecimal: ");
  Serial.println(numero, HEX);  // Exibe em hexadecimal

  Serial.println("Formato Octal: ");
  Serial.println(numero, OCT);  // Exibe em octal
}

void loop() {
  // Código adicional
}
```

Código-fonte 12 - Exemplo de como imprimir em diferentes bases o mesmo número
Fonte: Elaborado pelo autor (2024)

Saída esperada:

- Formato Decimal: 255
- Formato Binário: 11111111
- Formato Hexadecimal: FF
- Formato Octal: 377

4.3 Formatando números decimais (Float ou Double)

Para números com ponto flutuante (como “float” ou “double”), o “Serial.print()” também permite controlar o número de casas decimais a serem exibidas. Isso é útil quando estamos exibindo valores que requerem precisão, como medições de sensores.

Sintaxe: Serial.print(valor, numero_de_casas_decimais);

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Exemplo de código:

```
void setup() {
  Serial.begin(115200);
  float temperatura = 23.45678;

  Serial.print("Temperatura com 2 casas decimais: ");
  Serial.println(temperatura, 2);// Exibe com 2 casas decimais

  Serial.print("Temperatura com 4 casas decimais: ");
  Serial.println(temperatura, 4);// Exibe com 4 casas decimais
}

void loop() {
  // Código adicional
}
```

Código-fonte 13 - Exemplo de impressão de casas decimais
Fonte: Elaborado pelo autor (2024)

Saída esperada:

- Temperatura com 2 casas decimais: 23.46
- Temperatura com 4 casas decimais: 23.4568

4.4 Formatando strings

Já sabemos que strings são sequências de caracteres, e o ESP32 as manipula de forma simples com “Serial.print()”. Além de exibir uma string diretamente, podemos também combinar strings com números e variáveis em uma única linha, usando o operador “+” para concatenar. Neste exemplo, usamos a função “String()” para converter a variável “leitura” (um número inteiro) em uma string, permitindo que ela seja concatenada com outras strings.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
void setup() {
  Serial.begin(115200);

  String nome = "Sensor de Temperatura";
  int leitura = 25;

  // Exibindo uma string diretamente
  Serial.println("Bem-vindo ao Monitor Serial!");

  // Concatenando strings e variáveis
  Serial.println("Nome do dispositivo: " + nome);
  Serial.println("Leitura: " + String(leitura) + "°C");
}

void loop() {
  // Código adicional
}
```

Código-fonte 14 - Exemplo de impressão de strings combinadas
Fonte: Elaborado pelo autor (2024)

Saída esperada:

- Bem-vindo ao Monitor Serial!
- Nome do dispositivo: Sensor de Temperatura
- Leitura: 25°C

4.5 Formatação de múltiplos valores com uma única chamada

Podemos usar várias chamadas de “Serial.print()” em sequência para formatar múltiplos valores em uma única linha de saída. Aqui, usamos múltiplas chamadas de “Serial.print()” e “Serial.println()” para organizar a saída em uma única linha, com formatação personalizada para cada variável.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
void setup() {
  Serial.begin(115200);

  int sensor1 = 25;
  float sensor2 = 78.2345;

  Serial.print("Sensor 1: ");
  Serial.print(sensor1);
  Serial.print(" °C, Sensor 2: ");
  Serial.print(sensor2, 2);
  Serial.println(" % de umidade");
}

void loop() {
  // Código adicional
}
```

Código-fonte 15 - Exemplo de print() e println()
Fonte: Elaborado pelo autor (2024)

Saída esperada: Sensor 1: 25 °C, Sensor 2: 78.23 % de umidade

4.6 Alinhamento de colunas (Tabulação)

Se você precisa exibir dados em colunas alinhadas, pode usar o caractere de tabulação (“\t”) ou “espaços” para organizar a saída no Monitor Serial. Neste exemplo, usamos o caractere de tabulação “\t” para alinhar os dados em colunas, tornando a exibição mais organizada e fácil de entender.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
void setup() {
  Serial.begin(115200);

  Serial.println("Sensor\tTemperatura\tUmidade");
  Serial.print("1\t");
  Serial.print(25);
  Serial.print(" °C\t");
  Serial.println("60%");

  Serial.print("2\t");
  Serial.print(30);
  Serial.print(" °C\t");
  Serial.println("55%");
}

void loop() {
  // Código adicional
}
```

Código-fonte 16 - Exemplo de tabulação
Fonte: Elaborado pelo autor (2024)

Saída esperada:

Sensor	Temperatura	Umidade
1	25 °C	60%
2	30 °C	55%

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5 O USO DE DISPLAYS

O display LCD 16x2 é um módulo de exibição de informações que pode mostrar até 16 caracteres em duas linhas. Esse tipo de display é utilizado em projetos de eletrônica para exibir dados, mensagens, status do sistema e outros tipos de informações de forma clara e visível.

Quando conectado a um microcontrolador como o ESP32, o LCD 16x2 pode ser utilizado para fornecer feedback em tempo real, mostrando desde simples mensagens até dados processados por algoritmos de IA.

5.1 Funcionamento de um LCD 16x2

O LCD 16x2 possui 16 colunas e 2 linhas, permitindo exibir 32 caracteres no total. Cada caractere é formado por uma matriz de pontos de 5x8, o que possibilita a exibição de letras, números e alguns símbolos gráficos básicos. Esse display LCD 16x2 pode se comunicar com o microcontrolador de duas maneiras principais:

- Modo de 4 bits: utiliza 4 linhas de dados, economizando pinos no microcontrolador.
- Modo de 8 bits: utiliza 8 linhas de dados, proporcionando uma comunicação mais rápida, mas ocupando mais pinos.

Além dos pinos de dados, o LCD também requer pinos para controle, como RS (Register Select), E (Enable), e os pinos de alimentação (VCC e GND).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

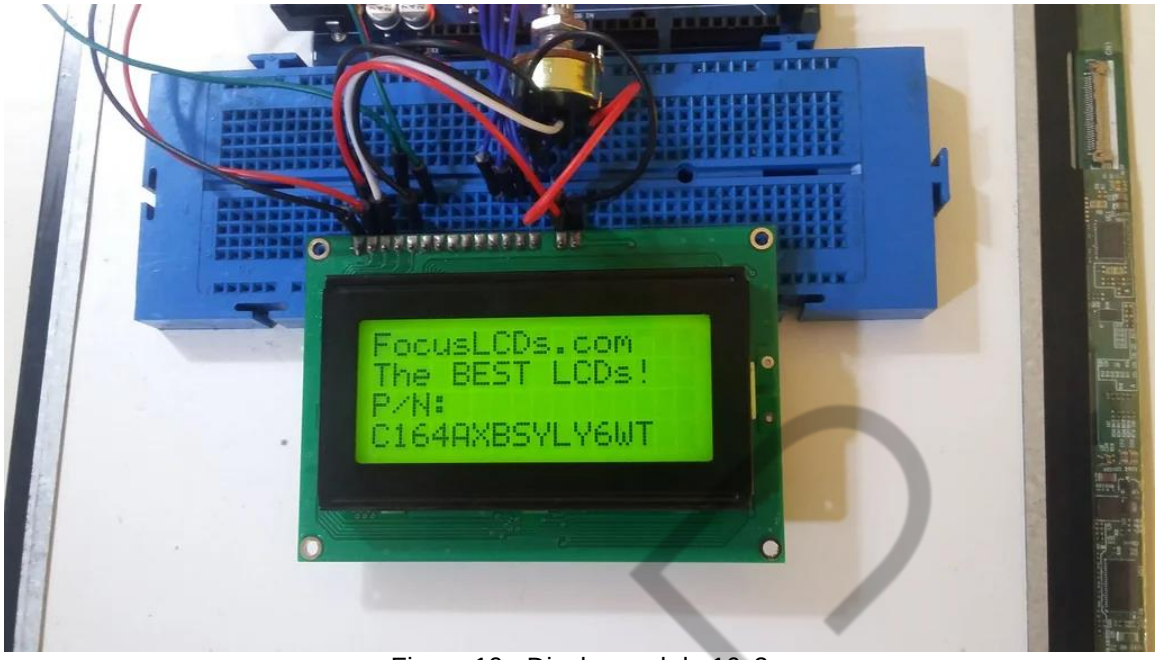


Figura 10 - Display real de 16x2
Fonte: Autodesk instructables (2018)

5.2 Pinagem e exemplo de código

Para conectar o LCD 16x2 ao ESP32 em modo de 4 bits, siga o esquema de conexão a seguir:

- SDA do módulo I2C conectado ao GPIO 21 do ESP32.
- SCL do módulo I2C conectado ao GPIO 22 do ESP32.
- VCC do módulo I2C conectado ao 3.3V ou 5V do ESP32 (dependendo do módulo).
- GND do módulo I2C conectado ao GND do ESP32.

Exemplo de código que utiliza a biblioteca "LiquidCrystal" para exibir uma mensagem no LCD 16x2 usando o ESP32:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// Endereço I2C do módulo do LCD, normalmente 0x27 ou 0x3F
#define I2C_ADDR 0x27
#define LCD_COLUMNS 16
#define LCD_ROWS 2

// Definindo os pinos SCL e SDA para o ESP32
#define I2C_SDA 21  // Pino GPIO 21 para SDA
#define I2C_SCL 22  // Pino GPIO 22 para SCL

// Cria um objeto para o LCD com as configurações do endereço e
dimensões
LiquidCrystal_I2C lcd(I2C_ADDR, LCD_COLUMNS, LCD_ROWS);

void setup() {
    // Inicializa a comunicação I2C com os pinos definidos
    Wire.begin(I2C_SDA, I2C_SCL);

    lcd.begin(LCD_COLUMNS, LCD_ROWS);  // Inicializa o LCD com
o número de colunas e linhas
    lcd.backlight();  // Liga a luz de fundo do LCD

    lcd.setCursor(0, 0);  // Posiciona o cursor na primeira
coluna da primeira linha
    lcd.print("Hello, I2C LCD!");  // Exibe a mensagem

    lcd.setCursor(0, 1);  // Posiciona o cursor na primeira
coluna da segunda linha
    lcd.print("ESP32 with I2C");  // Exibe outra mensagem
}

void loop() {
    // Aqui, o loop pode ser usado para atualizar o conteúdo do
LCD ou realizar outras tarefas
}
```

Código-fonte 17 - Exemplo de tabulação
Fonte: Elaborado pelo autor (2024)

Explicação do código:

- #include <Wire.h> esta biblioteca é necessária para comunicação I2C no ESP32. Ela permite que o ESP32 se comunique com dispositivos I2C, como o LCD.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- #include <LiquidCrystal_I2C.h> esta biblioteca facilita a interface com o display LCD utilizando o protocolo I2C, fornecendo funções de controle específicas para o LCD.
- I2C_ADDR define o endereço I2C do módulo LCD. Normalmente, os endereços mais comuns são “0x27” ou “0x3F”. Isso indica o endereço usado para se comunicar com o display via I2C.
- LCD_COLUMNS e LCD_ROWS definem as dimensões do display LCD (número de colunas e linhas). Neste caso, estamos configurando um display de 16 colunas e 2 linhas (16x2).
- I2C_SDA e I2C_SCL definem os pinos físicos do ESP32 que serão usados para os sinais SDA (Serial Data) e SCL (Serial Clock) do barramento I2C. Aqui, o código usa os pinos GPIO 21 para SDA e GPIO 22 para SCL, que são comuns para comunicação I2C no ESP32.
- LiquidCrystal_I2C lcd(I2C_ADDR, LCD_COLUMNS, LCD_ROWS); esta linha cria um objeto “lcd” do tipo LiquidCrystal_I2C, que será usado para controlar o display. O endereço I2C e as dimensões do display são passados como parâmetros. Esse objeto fornece funções como “lcd.print()”, “lcd.setCursor()”, e “lcd.backlight()” para controlar o LCD.
- Função setup(): é onde o LCD e a comunicação I2C são inicializados.
- Wire.begin(I2C_SDA, I2C_SCL); inicializa a comunicação I2C no ESP32, configurando os pinos SDA e SCL conforme definido anteriormente.
- lcd.begin(LCD_COLUMNS, LCD_ROWS); inicializa o LCD com as dimensões especificadas (16 colunas e 2 linhas), preparando-o para receber comandos e dados.
- lcd.backlight(); liga a luz de fundo do LCD para garantir que o conteúdo da tela seja visível.
- lcd.setCursor(0, 0); posiciona o cursor na primeira coluna da primeira linha do display.
- lcd.print("Hello, I2C LCD!"); exibe a mensagem "Hello, I2C LCD!" na primeira linha do display.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- lcd.setCursor(0, 1); posiciona o cursor na primeira coluna da segunda linha. Lembrando que 0,0 é o início de qualquer contagem em C/C++
- lcd.print("ESP32 with I2C"); exibe a outra mensagem
- A função loop() deve ficar vazia neste código, mas você pode usá-la para atualizar o conteúdo do LCD continuamente ou executar outras tarefas. É comum em projetos que requerem atualização dinâmica de dados (como a leitura de sensores ou cronômetros).

A figura “Tela do Wokwi solicitando a instalação da biblioteca I2C. Basta clicar no botão azul” está pedindo para que você confirme a instalação da biblioteca que gerencia o barramento de dados I2C (Inter-Integrated Circuit) que o display está usando para conversar com o ESP32.

I2C é um protocolo de comunicação serial que permite a troca de dados entre dispositivos, como microcontroladores (ESP32, Arduino) e periféricos (sensores, displays etc.), usando apenas dois fios: SDA (Serial Data Line) e SCL (Serial Clock Line).

- SDA: transmite os dados entre os dispositivos.
- SCL: fornece o sinal de relógio que sincroniza a transferência de dados.

O I2C é um barramento de comunicação que permite conectar múltiplos dispositivos (principal e secundários) no mesmo par de fios, sendo que cada dispositivo tem um endereço único. Um principal (como o ESP32) controla a comunicação, enquanto os secundários (como sensores ou displays) respondem aos comandos enviados pelo principal.

As vantagens do I2C é o uso de apenas dois fios, o que simplifica a conexão de múltiplos dispositivos, e permite a comunicação de até 127 dispositivos em um único barramento, isto é, um único par de SDA e SCL.

Essa simplicidade e versatilidade tornam o I2C amplamente utilizado em projetos de sistemas embarcados e IoT.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

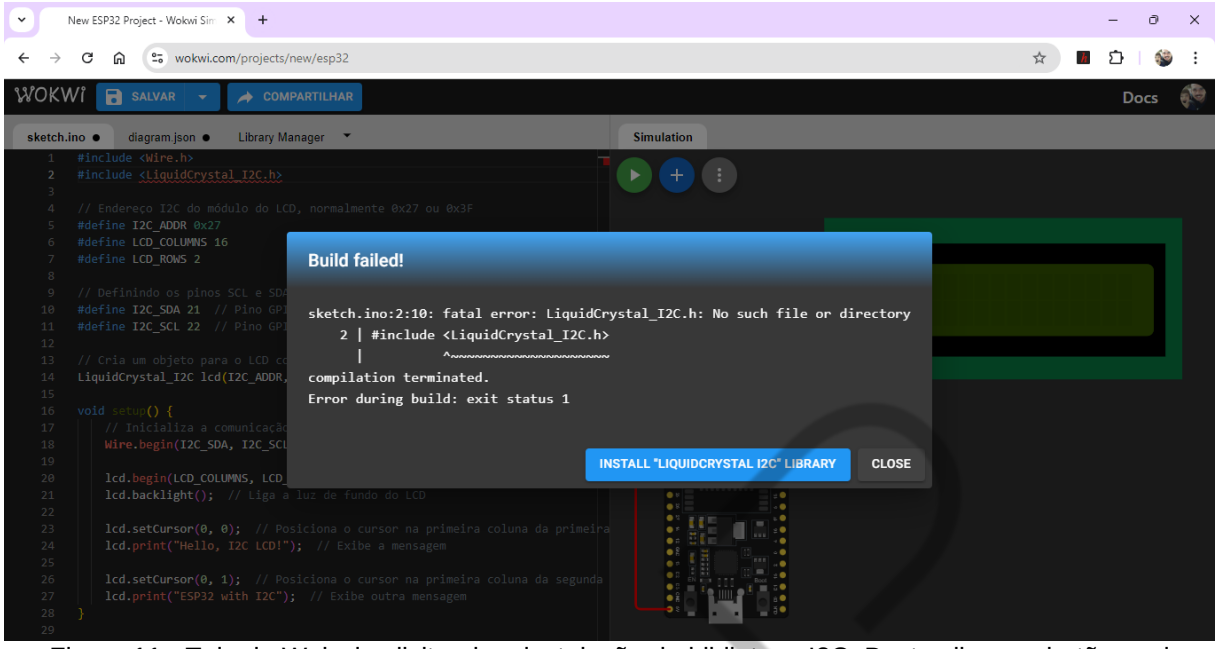


Figura 11 - Tela do Wokwi solicitando a instalação da biblioteca I2C. Basta clicar no botão azul
Fonte: Elaborado pelo autor (2024)

Depois que você confirmou a instalação, clique novamente na aba “sketch.ino” do seu projeto, e clique novamente em “play” para rodar seu programa.

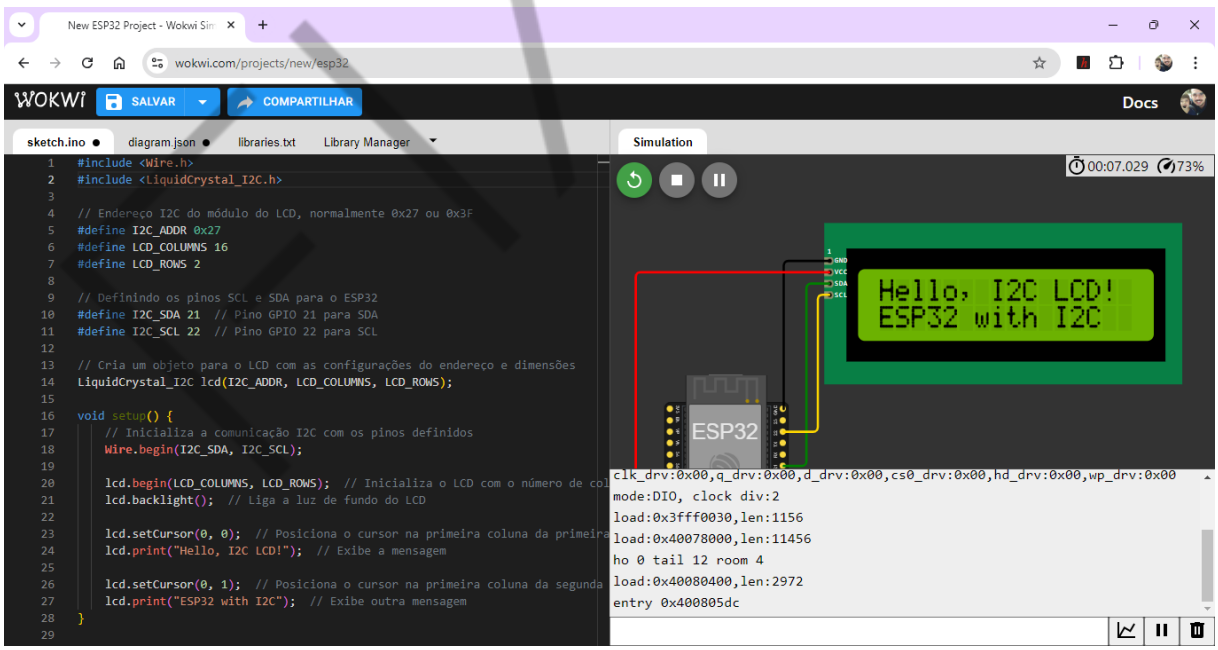


Figura 12 - Simulação do Wokwi usando LCD 16x2. Note que ele trabalha com 5V
Fonte: Elaborado pelo autor (2024)

Atenção: ao adicionar o LCD no Wokwi, busque por LCD e não display.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

A seguir, um último exemplo mostrando o mesmo LCD com atualizações dentro da função loop() fazendo papel de contador de 0 a 1000.

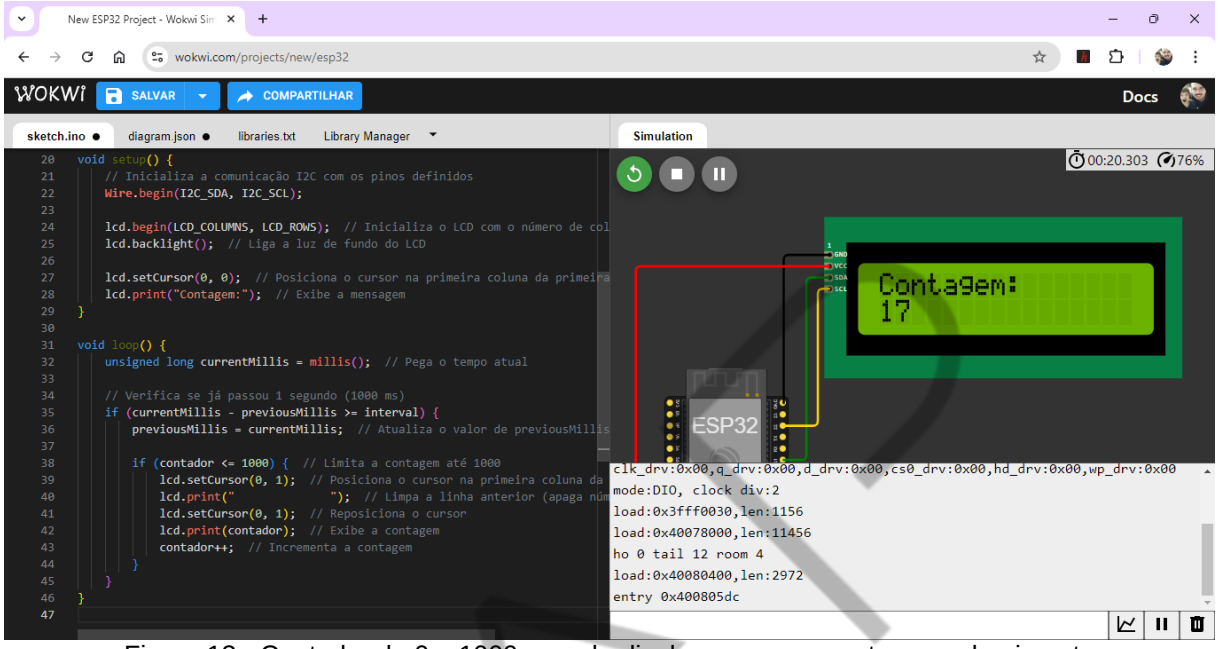


Figura 13 - Contador de 0 a 1000 usando display para apresentar o conhecimento
Fonte: Elaborado pelo autor (2024)

Código-fonte desse contador:

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// Endereço I2C do módulo do LCD, normalmente 0x27 ou 0x3F
#define I2C_ADDR 0x27
#define LCD_COLUMNS 16
#define LCD_ROWS 2

// Definindo os pinos SCL e SDA para o ESP32
#define I2C_SDA 21  // Pino GPIO 21 para SDA
#define I2C_SCL 22  // Pino GPIO 22 para SCL

// Cria um objeto para o LCD com as configurações do endereço e
dimensões
LiquidCrystal_I2C lcd(I2C_ADDR, LCD_COLUMNS, LCD_ROWS);

unsigned long previousMillis = 0;  // Armazena o último valor de
millis
const long interval = 1000;  // Intervalo de 1 segundo (1000 ms)
int contador = 0;  // Variável de contagem

void setup() {
    // Inicializa a comunicação I2C com os pinos definidos
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    Wire.begin(I2C_SDA, I2C_SCL);

    lcd.begin(LCD_COLUMNS, LCD_ROWS);  // Inicializa o LCD com
o número de colunas e linhas
    lcd.backlight();  // Liga a luz de fundo do LCD

    lcd.setCursor(0, 0);   // Posiciona o cursor na primeira
coluna da primeira linha
    lcd.print("Contagem:");  // Exibe a mensagem
}

void loop() {
    unsigned long currentMillis = millis();   // Pega o tempo
atual

    // Verifica se já passou 1 segundo (1000 ms)
    if (currentMillis - previousMillis >= interval) {
        previousMillis = currentMillis;  // Atualiza o valor de
previousMillis

        if (contador <= 1000) {  // Limita a contagem até 1000
            lcd.setCursor(0, 1);   // Posiciona o cursor na
primeira coluna da segunda linha
            lcd.print("                ");   // Limpa a linha
anterior (apaga números antigos aplicando espaços vazios)
            lcd.setCursor(0, 1);  // Reposiciona o cursor
            lcd.print(contador);  // Exibe a contagem
            contador++;  // Incrementa a contagem
        }
    }
}
```

Código-fonte 18 - Exemplo de tabulação
Fonte: Elaborado pelo autor (2024)

5.3 Aplicações do LCD 16x2 em sistemas de IA

O LCD 16x2 pode ser usado em diversas aplicações onde é necessário exibir informações ao usuário. Quando integrado a sistemas de IA, o LCD pode fornecer feedback em tempo real sobre o status do sistema, resultados de processamento, ou até mesmo interagir com o usuário. Exemplos:

- Em sistemas onde a IA processa dados em tempo real, como reconhecimento de imagem ou processamento de voz, o LCD pode exibir os resultados diretamente. Por exemplo, em um sistema de reconhecimento facial, o nome da pessoa identificada pode ser exibido na tela.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- Em sistemas de automação residencial, o LCD pode servir como uma interface simples para controlar dispositivos inteligentes. A IA pode ajustar automaticamente as configurações e exibir o status atual, como temperatura, umidade ou estado de luzes.
- Em robôs autônomos, o LCD pode exibir informações críticas, como coordenadas atuais, velocidade, ou status de sensores. A IA pode analisar esses dados e ajustar a navegação, enquanto o LCD fornece feedback ao operador.
- Em sistemas de segurança ou monitoramento, o LCD pode ser usado para notificar o usuário sobre eventos importantes, como a detecção de movimento ou a ativação de alarmes. A IA pode processar os dados de múltiplos sensores e exibir alertas detalhados.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

CONCLUSÃO

A representação do conhecimento no contexto de projetos com o ESP32 é uma das etapas mais importantes para garantir que os dados gerados e processados pelo microcontrolador sejam compreensíveis e úteis para o usuário final.

Em projetos que envolvem IoT e sistemas de controle e inteligência artificial, a capacidade de transformar dados brutos (como leituras de sensores, estados de dispositivos e resultados de processamento) em informações claras e legíveis faz toda a diferença no sucesso da aplicação.

Essa transformação de dados pode ser realizada por meio de dispositivos de exibição, como display LCD ou ainda via Monitor Serial, amplamente utilizado para depuração e interação com o sistema durante o desenvolvimento.

Os displays desempenham um papel crucial quando há necessidade de monitoramento visual em tempo real. Em projetos de IoT e IA, por exemplo, um display LCD pode mostrar informações importantes sobre o estado de um sistema, como a temperatura, a umidade, ou mesmo a condição de dispositivos controlados pelo ESP32.

Da mesma forma, o Monitor Serial, embora geralmente usado para depuração, permite a interação direta com o sistema, como a introdução de parâmetros ou comandos que alteram o comportamento do projeto, como ativar ou desativar sensores, ajustar valores de controle ou monitorar variáveis do sistema.

Outro aspecto importante é a formatação de dados exibidos, especialmente quando se lida com números, strings e leituras contínuas de sensores. Funções como “Serial.print()” e suas variantes permitem a apresentação personalizada dos dados, seja para exibir números inteiros, valores decimais com precisão ajustável ou até mesmo a formatação de mensagens de texto para torná-las mais informativas e organizadas.

Além disso, o uso de caracteres especiais e a habilidade de organizar dados em colunas no Monitor Serial facilitam a interpretação de informações complexas.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

A flexibilidade de entrada de dados pelo Monitor Serial também é uma vantagem durante o desenvolvimento. A possibilidade de ajustar parâmetros de forma dinâmica, sem a necessidade de recompilar o código ou reprogramar o dispositivo, torna o processo de desenvolvimento mais rápido e eficiente.

Isso é especialmente relevante em sistemas de IA, onde os parâmetros podem precisar de ajustes contínuos durante a fase de treinamento ou operação. Por exemplo, em um sistema de IA embarcado que aprende com dados de sensores, ajustar variáveis de controle em tempo real via Monitor Serial pode ser a chave para otimizar o desempenho.

Em termos de interação com o usuário, a representatividade dos dados também é essencial para garantir que o usuário final compreenda o que está acontecendo no sistema. A clareza das informações apresentadas, seja por meio de um display ou do Monitor Serial, pode impactar diretamente na usabilidade e eficácia do sistema. Um sistema bem projetado deve ser capaz de fornecer feedback rápido e preciso ao usuário, especialmente em projetos de automação ou monitoramento, onde decisões precisam ser tomadas com base nas informações apresentadas em tempo real.

Portanto, a representação do conhecimento no ESP32 não se trata apenas de exibir dados, mas de garantir que esses dados sejam relevantes, organizados e fáceis de interpretar. Isso resulta em sistemas mais eficientes, mais fáceis de usar e, consequentemente, mais confiáveis. Uma boa apresentação das informações contribui não apenas para o sucesso técnico de um projeto, mas também para a sua aceitação e adoção por parte dos usuários, especialmente em aplicações práticas no mundo da automação e inteligência artificial.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

ENGINEERLABS. How to Interface a 16x4 LCD With an Arduino. **Autodesk instructables**, 2018. Disponível em: <https://www.instructables.com/How-to-Interface-a-16x4-LCD-With-an-Arduino/>. Acesso em: 15 out. 2024.

ESPRESSIF. **Hardware Connection**. 2024. Disponível em: <https://docs.espressif.com/projects/esp-at/en/release-v3.0.0.0/esp32c2/Get_Started/Hardware_connection.html>. Acesso em: 10 out. 2024.

HONG, Chua Kiang *et al*. Analysis of Wind Speed Prediction using Artificial Neural Network and Multiple Linear Regression Model using Tinyml on Esp32. **Journal of Advanced Research in Fluid Mechanics and Thermal Sciences**, v. 107, n. 1, p. 29-44, 2023.

HURA, V.; MONASTYRSKII, L. IOT-based solution for detection of air quality using ESP32. **Artificial Intelligence**, n. 3, p. 86-93, 2023.

PASIC, Roberto; KUZMANOV, Ivo; ATANASOVSKI, Kokan. ESP-NOW communication protocol with ESP32. **Journal of Universal Excellence**, v. 6, n. 1, p. 53-60, 2021.

WAHYU, Yuyu *et al.* A Performance evaluation of ESP32 Camera Face Recognition for various projects. **Internet of Things and Artificial Intelligence Journal**, v. 2, n. 1, p. 10-21, 2022.

ZIM, Md Ziaul Haque. TinyML: analysis of Xtensa LX6 microprocessor for neural network applications by ESP32 SoC. arXiv preprint arXiv:2106.10652, 2021.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com